
PRUEBA TÉCNICA

Ingeniero de IA / Full Stack Senior – DomoNow

OFIMA SAS

Gracias por participar en nuestro proceso de selección.

Esta prueba busca evaluar principalmente tus capacidades como **Ingeniero de Software Senior**, especialmente en desarrollo **Full Stack con .NET, C# y Angular**, arquitectura de software, calidad, resolución de problemas y capacidad de utilizar herramientas de Inteligencia Artificial para mejorar el proceso de desarrollo.

Importante: esta no es una prueba de IA generativa ni esperamos que desarrolles un chatbot, un RAG o un agente de IA.

Nos interesa evaluar cómo utilizas la Inteligencia Artificial como una herramienta que apoye tu trabajo como ingeniero, sin reemplazar tu criterio técnico.

1. Contexto

DomoNow es una plataforma para la administración de propiedades horizontales.

Entre sus diferentes funcionalidades se encuentran procesos relacionados con residentes, unidades, zonas comunes, parqueaderos, visitantes, administración y otros servicios asociados a una propiedad.

Para esta prueba trabajaremos sobre un escenario simplificado del módulo de **Parqueaderos**.

Una propiedad residencial cuenta con parqueaderos de visitantes.

Actualmente, el registro y control de estos espacios requiere mejorar para permitir conocer su disponibilidad y evitar inconsistencias como asignar el mismo parqueadero a dos vehículos simultáneamente.

Tu objetivo será desarrollar una pequeña solución Full Stack que permita administrar estas asignaciones.

2. Caso de negocio

Se requiere construir una funcionalidad denominada:

Control de parqueaderos de visitantes

El sistema debe permitir que un usuario autorizado registre la utilización de un parqueadero por parte de un visitante.

Como mínimo se debe manejar la siguiente información:

- Parqueadero.
- Número o código del parqueadero.
- Estado del parqueadero.

-
- Placa del vehículo.
 - Nombre del visitante.
 - Unidad o apartamento que visita.
 - Fecha y hora de ingreso.
 - Fecha y hora de salida.
 - Estado de la asignación.

Los estados mínimos del parqueadero pueden ser:

- Disponible.
- Ocupado.
- Fuera de servicio.

Los estados mínimos de una asignación pueden ser:

- Activa.
- Finalizada.
- Cancelada.

3. Reglas de negocio

La solución debe contemplar como mínimo las siguientes reglas:

Regla 1

Un parqueadero **no puede tener más de una asignación activa simultáneamente**.

Regla 2

No se puede asignar un parqueadero cuyo estado sea **Fuera de servicio**.

Regla 3

Al registrar la salida de un vehículo, el parqueadero debe quedar nuevamente disponible.

Regla 4

La placa del vehículo es obligatoria y debe almacenarse de manera normalizada.

Por ejemplo:

abc - 123

debería almacenarse como:

ABC123

Regla 5

Una fecha de salida no puede ser anterior a la fecha de ingreso.

Regla 6

El sistema debe controlar correctamente situaciones de concurrencia.

Por ejemplo:

Dos usuarios intentan asignar el **Parqueadero 15** prácticamente al mismo tiempo.

El sistema debe garantizar que solamente una de las solicitudes pueda generar una asignación activa.

4. BLOQUE 1 – Backend

Tiempo sugerido: máximo 3 horas.

Construye una API utilizando:

- .NET / ASP.NET Core.
- C#.
- Entity Framework Core.
- SQL Server o PostgreSQL.

Puedes utilizar la versión estable de .NET con la que te sientas más cómodo.

Funcionalidades mínimas

Implementa endpoints que permitan:

Consultar parqueaderos

Obtener el listado de parqueaderos indicando:

- Código.
- Estado.
- Disponibilidad.

Opcionalmente puedes permitir filtros por estado.

Registrar ingreso

Crear una nueva asignación de parqueadero.

Ejemplo conceptual:

```
POST /api/parking-assignments
```

La solicitud podría contener:

```
{
  "parkingSpotId": 15,
  "licensePlate": "ABC123",
  "visitorName": "Carlos Pérez",
  "destinationUnit": "302"
}
```

Registrar salida

Finalizar una asignación activa.

Ejemplo conceptual:

```
PUT /api/parking-assignments/{id}/checkout
```

Consultar asignaciones activas

Permitir consultar todos los vehículos que actualmente se encuentran dentro de la propiedad.

5. Arquitectura

No existe una única arquitectura correcta.

Queremos conocer tu criterio.

Organiza el proyecto de una forma que consideres adecuada para una aplicación empresarial que potencialmente crecerá a cientos de funcionalidades.

Puedes utilizar, entre otros:

- Clean Architecture.
- Arquitectura Hexagonal.
- Vertical Slice Architecture.
- CQRS.
- Mediator.
- Repository Pattern.
- Domain Services.

No necesitas utilizar todos estos patrones.

Preferimos una arquitectura sencilla y bien justificada antes que una solución sobrearquitecturada.

6. Calidad del Backend

Esperamos encontrar:

- Separación adecuada de responsabilidades.
- Manejo adecuado de errores.
- Validaciones.
- Código mantenible.
- Uso adecuado de Dependency Injection.
- Uso correcto de operaciones asíncronas.
- Buen modelado de dominio.
- Manejo adecuado de concurrencia.
- Buenas prácticas de seguridad.
- Código comprensible.

7. Pruebas

Incluye pruebas automatizadas para las reglas de negocio que consideres más importantes.

Como mínimo queremos ver pruebas para:

1. Asignación correcta de un parqueadero disponible.
2. Intento de asignar un parqueadero ocupado.
3. Intento de asignar un parqueadero fuera de servicio.
4. Finalización de una asignación.

5. Validación de fechas.

Puedes utilizar:

- xUnit.
- NUnit.
- MSTest.

No buscamos una cobertura del 100%.

Nos interesa principalmente conocer **qué decides probar y por qué**.

8. BLOQUE 2 – Frontend

Tiempo sugerido: máximo 3 horas.

Construye una pequeña interfaz utilizando:

- Angular.
- TypeScript.

No evaluaremos diseño gráfico avanzado.

Nos interesa principalmente evaluar:

- Arquitectura frontend.
- Organización del código.
- Manejo de estados.
- Integración con APIs.
- TypeScript.
- Componentización.
- Experiencia básica de usuario.
- Manejo de errores.

9. Pantalla principal

Construye una pantalla denominada:

Parqueaderos de visitantes

Debe mostrar los parqueaderos mediante tarjetas, tabla o el componente que consideres adecuado.

Ejemplo:

PARQUEADEROS DE VISITANTES

P01 Disponible

P02 Ocupado ABC123

P03 Disponible

P04 Fuera de servicio

P05 Ocupado XYZ987

El usuario debe poder identificar rápidamente cuáles parqueaderos se encuentran:

- Disponibles.
- Ocupados.
- Fuera de servicio.

10. Registrar vehículo

Desde un parqueadero disponible debe ser posible registrar:

- Placa.
- Nombre del visitante.
- Unidad que visita.

Al guardar:

- Se debe consumir la API desarrollada.
- Actualizar la información presentada.
- Mostrar adecuadamente errores provenientes del backend.

11. Registrar salida

Desde un parqueadero ocupado debe existir una opción:

Registrar salida

Al ejecutarla:

- Se debe finalizar la asignación.
- El parqueadero debe regresar al estado disponible.
- La interfaz debe reflejar el cambio.

12. Inteligencia Artificial aplicada al desarrollo

En OFIMA promovemos el uso de Inteligencia Artificial como herramienta de ingeniería.

Durante esta prueba **puedes y recomendamos utilizar herramientas de IA** como:

- Cursor.
- GitHub Copilot.
- Claude Code.
- ChatGPT.
- Windsurf.
- Gemini.
- O cualquier otra herramienta que consideres útil.

No queremos evaluar si puedes desarrollar sin Inteligencia Artificial.

Queremos evaluar algo diferente:

Qué tan buen ingeniero eres utilizando Inteligencia Artificial.

Puedes utilizarla para:

- Generar código inicial.
- Analizar arquitectura.
- Crear pruebas.
- Refactorizar.
- Detectar errores.
- Analizar problemas de concurrencia.
- Documentar.
- Generar datos de prueba.
- Revisar código.
- Encontrar posibles vulnerabilidades.
- Analizar rendimiento.

13. Registro de utilización de IA

Incluye dentro del repositorio un archivo:

AI-USAGE.md

No es necesario copiar todas tus conversaciones.

Queremos que expliques brevemente entre **3 y 5 casos** donde utilizaste IA durante la prueba.

Para cada uno indica:

Problema

¿Qué necesitabas resolver?

Herramienta

¿Qué herramienta utilizaste?

Instrucción

Resume el prompt o instrucción utilizada.

Resultado

¿Qué propuso la IA?

Decisión del ingeniero

¿Qué aceptaste, modificaste o rechazaste?

Justificación

¿Por qué?

Ejemplo:

Problema:

Evitar asignaciones simultáneas del mismo parqueadero.

Herramienta:

Cursor.

Propuesta de IA:

Validar primero si existía una asignación activa.

Decisión:

Rechacé parcialmente la solución.

Motivo:

Una validación previa no garantiza consistencia ante solicitudes concurrentes.

Implementé adicionalmente un mecanismo de control a nivel de base de datos/transacción.

Este punto es especialmente importante.

Nos interesa conocer **tu criterio sobre lo producido por la IA**, no solamente comprobar que utilizaste una herramienta.

14. Revisión de código con IA

Una vez hayas terminado la solución, utiliza una herramienta de IA para realizar un **Code Review** de tu propio proyecto.

Pídele que busque al menos:

- Problemas de arquitectura.
- Posibles bugs.
- Problemas de concurrencia.
- Vulnerabilidades.
- Problemas de rendimiento.
- Código innecesariamente complejo.
- Ausencia de pruebas importantes.

Agrega en AI-USAGE.md:

AI Code Review

Incluye los **3 hallazgos más importantes** encontrados por la IA.

Para cada uno indica:

Hallazgo

¿Estás de acuerdo? Sí / No / Parcialmente

Acción tomada

Justificación técnica

No es obligatorio corregir todos los hallazgos.

También evaluaremos tu capacidad de explicar por qué **no realizarías una recomendación de la IA**.

15. Pregunta de arquitectura

Incluye en el README una sección:

Evolución de la solución

Responde brevemente:

Actualmente esta funcionalidad funciona para una sola propiedad con 30 parqueaderos.

¿Qué cambiarías si DomoNow tuviera que manejar 5.000 propiedades y cientos de miles de movimientos de parqueaderos diariamente?

No esperamos código adicional.

Nos interesa conocer tu razonamiento sobre temas como:

- Escalabilidad.
- Concurrencia.
- Caché.
- Bases de datos.
- Microservicios.
- Observabilidad.
- Mensajería.
- Seguridad.
- Multi-tenancy.
- Cloud.

No es obligatorio utilizar todos estos conceptos.

16. Desafío adicional – IA aplicada

Este punto es opcional.

No es necesario consumir OpenAI, Claude o ningún LLM.

Supongamos que DomoNow almacena millones de registros históricos de utilización de parqueaderos.

Explica brevemente cómo diseñarías una funcionalidad capaz de:

Predecir qué días y franjas horarias tendrán mayor demanda de parqueaderos de visitantes.

Puedes describir:

- Datos que utilizarías.
- Variables relevantes.
- Estrategia o algoritmo.
- Cómo entrenarías el modelo.
- Cómo medirías su efectividad.
- Cómo integrarías el resultado con .NET y Angular.
- Cómo llevarías el modelo a producción.
- Cómo detectarías degradación del modelo.

Máximo recomendado:

1 página.

No es obligatorio implementar el modelo.

17. Entregables

Entrega un repositorio Git con:

```
/backend  
/frontend  
README.md  
AI-USAGE.md
```

El README.md debe contener como mínimo:

- Cómo ejecutar el backend.
- Cómo ejecutar el frontend.
- Cómo configurar la base de datos.
- Decisiones principales de arquitectura.
- Supuestos realizados.
- Aspectos que mejorarías si tuvieras más tiempo.
- Respuesta a la pregunta de evolución de arquitectura.

Docker es bienvenido, pero no obligatorio.

18. Restricción de tiempo

Esta prueba está diseñada para realizarse en:

Máximo 6 horas.

Sugerimos dividirla en:

Bloque 1 – Backend: aproximadamente 3 horas.

Bloque 2 – Frontend, pruebas y revisión: aproximadamente 3 horas.

No esperamos un producto terminado para producción.

Preferimos una solución **incompleta pero técnicamente sólida y bien razonada** antes que una gran cantidad de funcionalidades generadas automáticamente sin suficiente criterio técnico.

Si no alcanzas a implementar alguna funcionalidad, puedes explicar en el README cómo la resolverías.

19. ¿Qué evaluaremos?

Backend y C# – 25%

- Diseño.
- Modelado.
- Calidad de código.
- APIs.
- Entity Framework.
- Manejo de errores.
- Concurrencia.

Arquitectura – 20%

- Separación de responsabilidades.
- SOLID.
- Escalabilidad.
- Decisiones y trade-offs.
- Capacidad de evitar sobreingeniería.

Angular y TypeScript – 20%

- Componentización.
- Servicios.
- Tipado.
- Integración con API.
- Organización.
- Manejo de errores.
- Experiencia de usuario.

Calidad y pruebas – 15%

- Pruebas relevantes.
- Legibilidad.
- Mantenibilidad.
- Git.
- Documentación.

Uso de Inteligencia Artificial – 15%

No evaluaremos quién utilizó más IA.

Evaluaremos:

- Calidad de las instrucciones.
- Capacidad de revisar resultados.
- Identificación de errores de la IA.

-
- Refactorización.
 - Uso de IA para testing y análisis.
 - Criterio técnico.

Comunicación técnica – 5%

- README.
- Explicación de decisiones.
- Claridad.
- Capacidad de justificar trade-offs.

20. Consideraciones finales

Puedes utilizar documentación, Internet y herramientas de Inteligencia Artificial.

No buscamos una solución idéntica a la arquitectura actual de DomoNow.

Tampoco existe una única respuesta correcta.

Buscamos conocer:

Cómo analizas un problema.

Cómo estructuras una solución.

Cómo escribes software.

Cómo pruebas lo que desarrollas.

Cómo utilizas Inteligencia Artificial.

Y, especialmente:

Cómo decides cuándo confiar y cuándo no confiar en lo que una herramienta de IA te propone.

Éxitos en la prueba.

OFIMA SAS